



UMCS

UNIwersytet Marii Curie-Skłodowskiej
w Lublinie

Wydział Matematyki, Fizyki i Informatyki

Kierunek: **informatyka**

Specjalność: **deweloper**

Patryk Wałach

nr albumu: 296597

Porównanie praktyk projektowania CSS: tradycyjne podejście z czystym CSS a nowoczesne narzędzia generujące CSS

Comparative study of CSS design practices:
traditional approach with pure CSS
versus modern CSS generating tools

Praca magisterska

napisana w Katedrze Cyberbezpieczeństwa i Lingwistyki Komputerowej
Instytutu Informatyki UMCS

pod kierunkiem **dr hab. Jarosława Byliny**

Lublin 2024

Spis treści

Wstęp	7
1 Ogólne wprowadzenie	9
1.1 Kaskadowe arkusze stylów	9
1.2 Metodologia <i>Blok Element Modyfikator</i>	9
1.3 Semantyczny CSS	10
1.4 Wszechstronny CSS	11
1.5 Statyczny CSS	11
1.6 Programowy CSS	11
2 Struktura HTML	13
2.1 Korzeń drzewa	13
2.2 Dziecko i Rodzic	14
2.3 Potomek i Przodek	14
2.4 Liść	14
2.5 Rodzeństwo	14
2.6 Sąsiadujące rodzeństwo	14
2.7 Młodsze rodzeństwo	14
3 Najczęściej wykorzystywane funkcjonalności języka CSS	17
3.1 Selektory	18
3.1.1 Selektor elementów	18
3.1.2 Selektor uniwersalny	18
3.1.3 Selektor id	19
3.1.4 Selektor klasy	19
3.1.5 Selektor grupujący	20
3.1.6 Selektor potomków	20
3.1.7 Selektor dziecka	20
3.2 Selektor młodszego rodzeństwa	21
3.3 Selektor sąsiadującego młodszego rodzeństwa	21
3.4 pseudoklasy	21

3.4.1	Funkcyjne pseudoklasy	22
3.5	pseudoelementy	22
3.6	Nadpisywanie stylów	23
3.7	Style warunkowe (ang. <i>media query</i>)	25
3.8	Zmienne CSS	26
4	Problemy języka CSS	27
4.1	Rosnące rozmiary plików	27
4.2	Kolizje pomiędzy stylami	27
4.2.1	Zmiana reguł bazowych	27
4.2.2	Kolizje nazw klas	28
4.3	Rozmieszczanie klas pomiędzy plikami CSS	28
5	Narzędzia	29
5.1	Język JavaScript	29
5.1.1	Środowisko Node.js	29
5.2	React	29
5.2.1	Nextjs	30
5.3	Storybook	30
5.3.1	Chromatic	30
6	Funkcjonalności bibliotek Tailwind CSS	31
6.1	Sposoby integracji	31
6.2	Warianty	31
6.3	Konfigurowalność	31
6.4	Dowolne wartości	32
6.5	Selektory	32
6.5.1	Selektor elementów, uniwersalny, id, klasy, grupujący	32
6.5.2	Selektor potomków	32
6.5.3	Selektor dziecka	33
6.6	Selektor młodszego rodzeństwa	33
6.7	Selektor sąsiadującego młodszego rodzeństwa	34
6.8	pseudoklasy i pseudoelementy	34
6.8.1	Funkcyjne pseudoklasy	34
6.9	Style warunkowe (ang. <i>media query</i>)	34
6.10	Zmienne CSS	34
6.11	Nadpisywanie stylów	35

7	Funkcjonalności biblioteki StyleX	37
7.1	Przestrzenie nazw	37
7.2	Selektory	37
7.3	pseudoklasy	38
7.4	pseudoelementy	38
7.5	Style warunkowe (ang. <i>media query</i>)	39
7.6	Zmienne CSS	39
7.7	Nadpisywanie stylów	40
8	Co zostało zrobione?	41
8.1	Projekt strony	41
8.1.1	Wpisy	41
8.1.2	Formularze	44
8.2	Implementacja strony	45
8.3	Refaktoryzacja strony	47
9	Wnioski	49
9.1	Zmienne	49
9.2	Kolokacja kodu	52
9.3	Prefiksy	53
9.4	De duplikacja kodu	54
9.5	Nadpisywanie stylów	55
9.5.1	Porównanie kryteriów	55
	Podsumowanie	57
	Bibliografia	59

Wstęp

Język CSS jest nierozłączną częścią stron internetowych. Wraz z nieustannym rozwojem internetu i rosnącym rozmiarem stron internetowych nasuwają się pytania, w jaki sposób tworzyć łatwy w utrzymaniu, czytelny i optymalny kod CSS. Dlatego w niniejszej pracy sprawdzimy jak z tymi problemami radzi sobie tradycyjne podejście z czystym CSS i nowoczesne narzędzia generujące CSS.

W pierwszym rozdziale przedstawimy podstawowe pojęcia związane z CSS. Następnie przedstawimy podstawy struktury HTML. W kolejnym spoglądamy na najczęściej wykorzystywane funkcje CSS. W czwartym rozdziale opisujemy problemy języka CSS. W rozdziale piątym przedstawimy narzędzia, których użyliśmy w celu zbudowania strony internetowej. Następnie prezentujemy jak wyglądają podstawowe funkcje bibliotek Tailwind CSS i StyleX. W ostatnim rozdziale przedstawiamy wnioski z napisanego przez nas kodu.

Rozdział 1

Ogólne wprowadzenie

1.1 Kaskadowe arkusze stylów

Kaskadowe arkusze stylów (ang. *Cascading Style Sheets*, w skrócie CSS) jest to deklaratywny język służący do kontrolowania prezentacji dokumentów HTML, XML, SVG itp.

Historia CSS sięga lat 90. XX wieku, kiedy to został stworzony jako odpowiedź na potrzebę oddzielenia struktury dokumentu od jego stylizacji. Wcześniej wszystkie style były zdefiniowane bezpośrednio w kodzie HTML, co sprawiało, że utrzymanie i zarządzanie stronami internetowymi było trudne i niewydajne.

Za pomocą CSS można kontrolować różne aspekty prezentacji, takie jak kolory, czcionki, marginesy, odstępy między elementami, rozmiar i położenie elementów na stronie, animacje itp.

Dla przykładu poniższy kod zmienia kolor akapitu na czerwony:

```
1 p {  
2   color: red;  
3 }
```

Język CSS jest stale rozwijany, co oznacza, że pojawiają się nowe funkcje i możliwości, które pozwalają projektantom jeszcze lepiej kontrolować wygląd swoich stron internetowych. Istnieją również różne wersje CSS, z których najnowsza, CSS3, wprowadza wiele nowych funkcji, takich jak efekty przejścia, animacje, gradienty czy elastyczne rozkładanie elementów.

1.2 Metodologia *Blok Element Modyfikator*

Blok Element Modyfikator (w skrócie BEM) jest to metodologia ułatwiająca tworzenie komponentów wielokrotnego użytku.

Historia BEM sięga 2009 roku, kiedy została wprowadzona przez zespół deweloperów w Mail.Ru, jednej z największych rosyjskich firm internetowych. Jej celem było rozwiązanie problemów związanych z rosnącą złożonością stron internetowych i aplikacji oraz utrzymaniem spójności i czytelności kodu CSS.

Zgodnie z metodologią BEM nazwa każdej z klas CSS musi składać się z trzech elementów:

- Blok (ang. *Block*) - niezależny komponent, który stanowi podstawową część struktury interfejsu. Blok może zawierać inne bloki lub elementy. Każdy blok ma unikalną nazwę, która jest używana w klasach CSS do identyfikacji jego stylów, np. `.header`, `.container`, `.menu`, `.checkbox`, `.input`.
- Element (ang. *Element*) - część składowa bloku, która ma znaczenie tylko w kontekście tego bloku. Elementy są zależne od bloku, w którym się znajdują, i nie mogą istnieć samodzielnie poza nim. Nazwy elementów są prefiksowane nazwą bloku, do którego należą, i oddzielone dwoma podkreślnikami, np. `.menu__item`, `.list__item`, `.checkbox__caption`, `.header__title`.
- Modyfikator (ang. *Modifier*) - odpowiada za zmianę wyglądu lub zachowania bloku lub elementu. Dzięki modyfikatorom możliwe jest dostosowywanie wyglądu i funkcjonalności istniejących bloków i elementów bez konieczności tworzenia nowych. Nazwy modyfikatorów są prefiksowane nazwą bloku lub elementu, do którego się odnoszą, i oddzielone dwoma minusami, np. `.btn--disabled`, `.link--highlighted`, `.checkbox--checked`, `.nav--fixed`, `.nav__link--size-big`, `.nav__link--color-yellow`.

1.3 Semantyczny CSS

Semantyczny CSS jest to metodologia, której jednym z największych priorytetów jest *separacja obowiązków* [6].

Idea ta zakłada, że HTML powinien zawierać tylko informacje dotyczące treści, a wszystkie decyzje dotyczące stylizacji powinny być podejmowane w kodzie CSS.

Dla przykładu:

```
1 <p class="text-center">
2   Hello world!
3 </p>
```

Kod zawiera jedną klasę `.text-center`. Centrowanie tekstu to decyzja stylistyczna, więc ten kod łamie zasadę *separacji obowiązków*, ponieważ pozwoliliśmy, aby informacje dotyczące stylizacji przeniknęły do naszego kodu HTML.

Zamiast tego, zalecane podejście polega na nadawaniu elementom nazw klas na podstawie ich zawartości i wykorzystaniu tych klas jako punktów zaczepienia w kodzie CSS do stylizacji kodu:

```
1  <style>
2  .greeting {
3      text-align: center;
4  }
5  </style>
6
7  <p class="greeting">
8      Hello world!
9  </p>
```

1.4 Wszechstronny CSS

Wszechstronny CSS (ang. *utility-first CSS*) to podejście odwrotne do Semantycznego CSS. Skupia się na tworzeniu małych, pojedynczych klas CSS, z których każda odpowiada za jedną, konkretną właściwość stylu. Zamiast tworzyć specjalne klasy dla konkretnych elementów interfejsu użytkownika, jak nagłówki, przyciski itp. używa się klas, które definiują pojedyncze właściwości CSS, takie jak kolor, margines czy wypełnienie.

1.5 Statyczny CSS

Statyczny CSS to metodologia, w której kod CSS nie zmienia się w trakcie działania aplikacji. Kod CSS jest pisany wprost lub generowany na etapie kompilacji po przeanalizowaniu kodu. Tworząc kod w ten sposób, możemy skorzystać z wachlarza narzędzi, analizujących i modyfikujących kod CSS w celu znalezienia błędów, zminimalizowania lub polyfillowania kodu¹.

1.6 Programowy CSS

Programowy CSS to z kolei metodologia, w której style CSS są tworzone w trakcie działania aplikacji. Przewagą tego rozwiązania nad statycznym CSS przez wiele lat była możliwość generowania stylów dynamicznych. Zastosowanie tej metodologii nie wymaga szerokiej znajomości języka CSS – style zwykle definiowane są przy użyciu bibliotek, które zapewniają wysoko poziomowy interfejs. Wykorzystując tego typu biblioteki w językach

¹polyfillowania kodu – zastąpienia kodu niewspieranego przez niektóre przeglądarki równoważnym działającym kodem [1]

z silnym typowaniem, możemy automatycznie korzystać z auto uzupełniania kodu i raportowania błędów w edytorze bez dodatkowych narzędzi, wymaganych do zrozumienia składni języka CSS.

Rozdział 2

Struktura HTML

Struktura HTML może być reprezentowana jako drzewo, co jest przydatne do wizualizacji hierarchii elementów oraz do manipulacji dokumentem w języku JavaScript. To drzewo nazywane jest drzewem DOM (ang. *Document Object Model*). Na listingu 2.1 widzimy przykład kodu HTML. Kod ten można przedstawić jako drzewo.

Listing 2.1: Przykładowy kod HTML

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <title>Struktura HTML</title>
5  </head>
6  <body>
7      <h1>Witamy!</h1>
8      <p>To jest akapit.</p>
9      <div>
10         <span>Kolejny akapit w elemencie div.</span>
11     </div>
12 </body>
13 </html>
```

2.1 Korzeń drzewa

Korzeń drzewa, to węzeł, który jest na szczycie hierarchii drzewa i od którego wszystkie inne węzły (elementy) są bezpośrednio lub pośrednio zależne. Korzeń jest jedynym węzłem w drzewie, który nie ma rodzica. Wszystkie inne węzły w drzewie są jego potomkami. W listingu 2.1 widzimy przykład drzewa, w którym element `<html>` jest korzeniem drzewa.

2.2 Dziecko i Rodzic

Dziecko (ang. *child*) jest węzłem, który jest bezpośrednio podłączony do innego węzła, zwanego jego rodzicem (ang. *parent*). Węzeł rodzic może mieć wiele dzieci, ale każde dziecko ma tylko jednego rodzica. W listingu 2.1 widzimy przykład drzewa, w którym element `<title>` jest dzieckiem elementu `<head>`.

2.3 Potomek i Przodek

Potomek (ang. *descendant*) jest węzłem, który znajduje się w hierarchii drzewa poniżej innego węzła, zwanego przodkiem (ang. *ancestor*). W listingu 2.1 widzimy przykład drzewa, w którym element `<span>` jest potomkiem elementu `<body>`.

2.4 Liść

Liść (ang. *leaf*) jest węzłem, który nie ma żadnych dzieci. Są to elementy najniżej położone w hierarchii drzewa. W listingu 2.1 widzimy przykład drzewa, w którym tekst "Witamy!" jest liściem.

2.5 Rodzeństwo

Rodzeństwo (ang. *sibling*) odnosi się do węzłów, które mają tego samego rodzica. Rodzeństwo to elementy znajdujące się na tym samym poziomie hierarchii drzewa, bezpośrednio obok siebie. W listingu 2.1 widzimy przykład drzewa, w którym elementy `<p>`, `<div>`, `<h1>` są rodzeństwem.

2.6 Sąsiadujące rodzeństwo

Sąsiadujące rodzeństwo (ang. *adjacent siblings*) odnosi się do dwóch węzłów w strukturze drzewa, które mają tego samego rodzica i sąsiadują ze sobą bezpośrednio na tym samym poziomie hierarchii, nie mając żadnego innego węzła pomiędzy nimi na tym samym poziomie hierarchii. W listingu 2.1 widzimy przykład drzewa, w którym element `<h1>` ma tylko jedno sąsiadujące rodzeństwo element `<p>`.

2.7 Młodsze rodzeństwo

Młodsze rodzeństwo (ang. *younger sibling*) jest węzłem, który znajduje się na tym samym poziomie hierarchii co inny węzeł, ale jest umieszczony po nim. W praktyce oznacza

to, że oba węzły mają tego samego rodzica, ale młodsze rodzeństwo występuje później w kolejności. W listingu [2.1](#) widzimy przykład drzewa, w którym element `<p>` i `<div>` są młodszym rodzeństwem elementu `<h1>`.

Rozdział 3

Najczęściej wykorzystywane funkcjonalności języka CSS

Składnia arkusza stylów CSS przedstawiona jest na listingu 3.1. Jednym z głównych elementów tej składni są zestawy reguł¹, których składnia przedstawiona jest na listingu 3.2. Na rysunku 3.1 widzimy przykład takiej reguły, składającej się z selektora elementu `p` oznaczonego niebieskim kolorem, oraz zbioru deklaracji, znajdującego się pomiędzy nawiasami. W zbiorze deklaracji na tym rysunku widzimy tylko jedną deklarację `color: red;` składającą się z atrybutu oznaczonego kolorem czerwonym, dwukropka, wartości atrybutu i średnika. Każda reguła musi składać się z co najmniej jednej deklaracji.

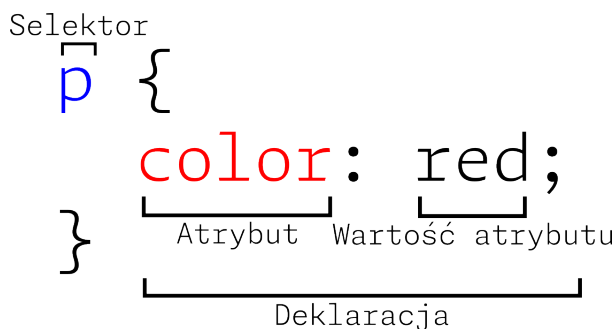
Listing 3.1: Składnia arkusza stylów CSS [4]

```
1 stylesheet
2   : [ CHARSET_SYM STRING ';' ]?
3     [S|CDO|CDC]* [ import [ CDO S* | CDC S* ]* ]*
4     [ [ ruleset | media | page ] [ CDO S* | CDC S* ]* ]*
5   ;
```

Listing 3.2: Składnia reguły CSS [4]

```
1 ruleset
2   : selector [ ',' S* selector ]*
3     '{' S* declaration? [ ';' S* declaration? ]* '}' S*
4   ;
5
6 declaration
7   : property ':' S* expr prio?
8   ;
```

¹zestaw reguł – Termin zestaw reguł jest często nazywany po prostu regułą



Rysunek 3.1: Przykład reguły CSS, zmieniający kolor akapitu na czerwony

3.1 Selektory

Listing 3.3: Składnia selektora CSS [4]

```
1 selector
2   : simple_selector [ combinator selector | S+ [ combinator? selector ]? ]?
3   ;
```

Listing 3.4: Składnia prostego selektora CSS [4]

```
1 simple_selector
2   : element_name [ HASH | class | attrib | pseudo ]*
3   | [ HASH | class | attrib | pseudo ]+
4   ;
```

3.1.1 Selektor elementów

Listing 3.5: Składnia selektora elementów CSS [4]

```
1 element_name
2   : IDENT | '*'
3   ;
```

Selektor elementów wybiera elementy HTML na podstawie nazwy elementu. Dla przykładu listing 3.6.

Listing 3.6: Reguła zostanie zastosowana do wszystkich akapitów na stronie

```
1 p {
2   color: red;
3 }
```

3.1.2 Selektor uniwersalny

Selektor uniwersalny wybiera wszystkie elementy HTML na stronie. Dla przykładu listing 3.7.

Listing 3.7: Reguła zostanie zastosowana do wszystkich elementów na stronie

```
1  * {  
2    color: red;  
3  }
```

3.1.3 Selektor id

Listing 3.8: Składnia tokenu HASH zwracanego przez lekser CSS [4]

```
1  "#" {name}
```

Selektor id wybiera elementy HTML na podstawie atrybutu id elementu. Id elementu jest unikalne na stronie, dlatego selektor id służy do wyboru jednego unikalnego elementu. Dla przykładu listing 3.9.

Listing 3.9: Reguła zostanie zastosowana do elementu HTML o atrybucie id="para1"

```
1  #para1 {  
2    color: red;  
3  }
```

3.1.4 Selektor klasy

Listing 3.10: Składnia selektora klasy CSS [4]

```
1  class  
2  : '.' IDENT  
3  ;
```

Selektor klasy wybiera elementy HTML na podstawie atrybutu class elementu. Dla przykładu listing 3.11.

Listing 3.11: Reguła zostanie zastosowana do elementów HTML o atrybucie class="text-red"

```
1  .text-red {  
2    color: red;  
3  }
```

Selektor klasy może być używany w połączeniu z selektorem elementu. Dla przykładu listing 3.12.

Listing 3.12: Reguła zostanie zastosowana do akapitów o atrybucie class="text-red"

```
1  p.text-red {  
2    color: red;  
3  }
```

Elementy HTML mogą posiadać wiele klas. Dla przykładu listing 3.13.

Listing 3.13: Element będzie stylizowany przez reguły `.center` i `.large`

```
1 <p class="center large"></p>
```

3.1.5 Selektor grupujący

Selektor grupujący pozwala na połączenie wielu takich samych reguł. Dla przykładu niegrupowane reguły na rysunku 3.14 można zastąpić równoważną regułą tak jak na rysunku 3.15.

Listing 3.14: Niegrupowane reguły

```
1 h1 {  
2   color: red;  
3 }  
4 h2 {  
5   color: red;  
6 }  
7 p {  
8   color: red;  
9 }
```

Listing 3.15: Pogrupowane reguły

```
1 h1, h2, p {  
2   color: red;  
3 }
```

3.1.6 Selektor potomków

Selektor potomków dopasowuje wszystkie elementy będące potomkami określonego elementu. Dla przykładu listing 3.16.

Listing 3.16: Reguła zostanie zastosowana do akapitów, które są potomkami sekcji

```
1 section p {  
2   color: blue;  
3 }
```

3.1.7 Selektor dziecka

Selektor dziecka wybiera wszystkie elementy będące dziećmi określonego elementu. Dla przykładu listing 3.17.

Listing 3.17: Reguła zostanie zastosowana do akapitów, które są dziećmi sekcji

```
1 section > p {
```

```
2     color: blue;  
3 }
```

3.2 Selektor młodszego rodzeństwa

Selektor młodszego rodzeństwa wybiera wszystkie elementy będące młodszym rodzeństwem określonego elementu. Dla przykładu listing 3.18.

Listing 3.18: Reguła zostanie zastosowana do wszystkich akapitów, umieszczonych za sekcją

```
1     section ~ p {  
2         color: blue;  
3     }
```

3.3 Selektor sąsiadującego młodszego rodzeństwa

Selektor sąsiadującego młodszego rodzeństwa wybiera element będący sąsiadującym młodszym rodzeństwem określonego elementu. Dla przykładu listing 3.19.

Listing 3.19: Reguła zostanie zastosowana do akapitów, bezpośrednio poprzedzonych sekcją

```
1     section + p {  
2         color: blue;  
3     }
```

3.4 pseudoklasy

Listing 3.20: Składnia pseudoklasy CSS [4]

```
1 pseudo  
2 : ':' ' [ IDENT | FUNCTION S* [IDENT S*]? ' )' ' ]  
3 ;
```

pseudoklasy opisują stany elementów. Na przykład, mogą być użyte do:

- Stylizacji elementu, gdy użytkownik najedzie na niego myszką (listing 3.21)
- Stylizacji odwiedzonych i nieodwiedzonych linków w różny sposób (listing 3.22)
- Stylizacji elementu, gdy otrzymuje skupienie ² (listing 3.23)

²element otrzymujący skupienie to element który, będzie jako pierwszy informowany o zdarzeniach, np. o wciśnięciu klawiatury

Listing 3.21: Reguła zostanie zastosowana do akapitu, na który użytkownik najedzie myszką

```
1   p:hover {  
2       color: blue;  
3   }
```

Listing 3.22: Reguły zmieniają kolor linku zależnie od jego stanu

```
1   a:link {  
2       color: red;  
3   }  
4   a:visited {  
5       color: blue;  
6   }
```

Listing 3.23: Reguła zmienia kolor przycisku, gdy otrzymuje skupienie, np. poprzez wciśnięcie klawisza **Tab**

```
1   button:focus {  
2       color: red;  
3   }
```

3.4.1 Funkcyjne pseudoklasy

Listing 3.24: Składnia tokena FUNCTION zwracanego przez lekser CSS [4]

```
1   {ident}" ("
```

Funkcyjne pseudoklasy to klasy, które akceptują listę selektorów jako parametry. Dla przykładu listing 3.25.

Listing 3.25: Reguła zostanie zastosowana do elementów, które nie są elementami `div` i nie są elementami o atrybucie `class="fancy"`

```
1   :not(div):not(.fancy) {  
2       color: blue;  
3   }
```

3.5 pseudoelementy

pseudoelementy służą do stylizacji określonych części elementów.

Na przykład, mogą być używane do:

- Stylizacji pierwszej litery lub linii elementu (listing 3.26)
- Wstawienia treści przed lub po treści elementu (listing 3.27)

Listing 3.26: Reguła zostanie zastosowana do pierwszej litery każdego akapitu

```
1 p::first-letter {  
2   color: red;  
3 }
```

Listing 3.27: Reguła doda napis "Hello" do każdego akapitu

```
1 p::before {  
2   content: "Hello";  
3 }
```

3.6 Nadpisywanie stylów

Jeśli istnieją dwie lub więcej reguł CSS wskazujących na ten sam element, selektor o najwyższej specyficzności "wygra", i jego deklaracja stylu zostanie zastosowana do tego elementu HTML. Dla przykładu listing 3.28.

Listing 3.28: Tekst akapitu będzie zielony, ponieważ reguła `.test` ma najwyższą specyficzność

```
1 <html>  
2 <head>  
3   <style>  
4     .test {color: green;}  
5     p {color: red;}  
6   </style>  
7 </head>  
8 <body>  
9 <p class="test">Hello World!</p>  
10 </body>  
11 </html>
```

Każdy selektor CSS ma swoje miejsce w hierarchii specyficzności.

Istnieją cztery kategorie, które określają poziom specyficzności selektora:

- style liniowe – specyficzność 1000, na przykład: `<h1 style="color: pink;">`;
- identyfikatory – specyficzność 100, na przykład: `#navbar`;
- klasy, pseudoklasy, selektory atrybutów – specyficzność 10, na przykład: `.test`, `:hover`, `[href]`;
- elementy i pseudoelementy – specyficzność 1, na przykład: `h1`, `::before`.

Dodatkowo, selektor uniwersalny posiada specyficzność 0.

Znaczenie też ma kolejność reguł. Reguły definiowane później w arkuszu CSS mają potencjalnie możliwość nadpisania wcześniejszych reguł, pod warunkiem, że mają równą lub wyższą specyfikę.

Tabela 3.1 pokazuje przykłady obliczenia specyficzności.

Tabela 3.1: Przykłady obliczenia specyficzności

Selektor	Specyficzność	Kalkulacja
*	0	0
a	1	1
.btn	10	10
#demo	100	100
<p style="color: pink;">	1000	1000
a.link	11	1 + 10
a.btn.blue	21	1 + 10+10
section#demo	101	1 + 100

Dodatkowo deklaracja `!important` może być użyta, by nadpisać reguły, niezależnie od ich specyficzności. Jeżeli istnieje więcej niż jedna reguła z deklaracją `!important` to reguła z największą specyficznością jest nadpisywana. Na listingu 3.29 widzimy, że reguła `#submit.btn` z większą specyficznością jest nadpisywana przez regułę `.btn--red` z mniejszą specyficznością.

Listing 3.29: Przykład użycia deklaracji `!important`

```

1  #submit.btn {
2      color: blue;
3  }
4  .btn--red {
5      color: red !important;
6  }
7
```

Listing 3.30: Składnia priorytetu CSS [4]

```

1  prio
2  : IMPORTANT_SYM S*
3  ;
```

Listing 3.31: Składnia symbolu `!important` w CSS [4]

```

1  "!"({w}|{comment})*{I}{M}{P}{O}{R}{T}{A}{N}{T} {return IMPORTANT_SYM;}
```

3.7 Style warunkowe (ang. *media query*)

Style warunkowe, są kluczowe dla responsywnego projektowania interfejsów. Pozwalają one dostosować wygląd stron internetowych do różnych urządzeń i warunków przeglądania. Składnia styli warunkowych w kodzie CSS jest opisana na listingach 3.32 i 3.33.

Listing 3.32: Składnia styli warunkowych [4]

```
1 media
2   : MEDIA_SYM S* media_list '{' S* ruleset* '}' S*
3   ;
4 media_list
5   : medium [ COMMA S* medium]*
6   ;
7 medium
8   : IDENT S*
9   ;
```

Listing 3.33: Składnia tokena MEDIA_SYM zwracanego przez lekser CSS [4]

```
1 @{M}{E}{D}{I}{A}
```

Przykładami wykorzystania stylów warunkowych są:

- zmiana stylów zależnie od szerokości ekranu – listing 3.34,
- ograniczenie animacji zgodnie z preferencją użytkownika – listing 3.35.

Listing 3.34: Kod CSS zmieniający szerokość elementu zależnie od szerokości ekranu

```
1 .container {
2   width: 100%;
3 }
4
5 @media (min-width: 768px) {
6   .container {
7     width: 75%;
8   }
9 }
```

Listing 3.35: Kod CSS ograniczający animacje zgodnie z preferencją użytkownika

```
1 @media (prefers-reduced-motion: no-preference) {
2   .spin {
3     animation: spin 1s linear infinite;
4   }
5 }
```

3.8 Zmienne CSS

Zmienne mogą przechowywać dowolne wartości, takie jak kolory, rozmiary czcionek, marginesy itp. zmienne dostępne są dla wszystkich potomków elementu, dla którego zostały utworzone.

Przykładem użycia zmiennych jest listing 3.36, gdzie zdefiniowano kilka podstawowych kolorów, które będą używane w różnych miejscach na stronie. Tego typu kod pozwala na łatwą zmianę kolorów całej strony lub tylko jej części.

Listing 3.36: Zmienne CSS

```
1 :root {  
2   --color-primary: #007bff;  
3   --color-secondary: #6c757d;  
4 }  
5  
6 .alert {  
7   background-color: var(--accent-color);  
8   color: var(--primary-color);  
9 }
```

Rozdział 4

Problemy języka CSS

4.1 Rosnące rozmiary plików

Im większy plik CSS, tym dłużej zajmie mu załadowanie się na stronie internetowej. Duży rozmiar plików może powodować spowolnienie czasu ładowania strony, co z kolei może prowadzić do niższej oceny strony w wynikach wyszukiwania i gorszego doświadczenia użytkownika.

W sytuacji, kiedy wielu programistów równolegle ręcznie tworzy kod CSS, dochodzi do powtórzeń kodu. Tego typu praca wymaga regularnej optymalizacji i refaktoryzacji kodu w celu zmniejszenia jego rozmiaru. Proces ten wraz z czasem staje się coraz dłuższy i droższy ze względu na rosnący rozmiar kodu.

Na rosnący rozmiar kodu wpływ ma też niewykorzystywany kod CSS. Wraz ze zmianą struktury HTML, część stylów staje się nieużywana. Style te często nie są łatwe do znalezienia, ze względu na złożoność zastosowanych selektorów i trudność analizy kodu HTML, np. struktura HTML zmienia się pod wpływem kodu JavaScript.

4.2 Kolizje pomiędzy stylami

4.2.1 Zmiana reguł bazowych

Deweloperzy muszą używać elementów HTML do pisania HTML, a liczba elementów, które można wybrać, jest ograniczona. Dlatego może to być kuszące, aby zdefiniować wiele podstawowych stylów za pomocą selektorów elementów, aby uniknąć dodawania klas do wszystkich elementów, to takie podejście tworzy niezadeklarowaną zależność między tymi regułami a wszystkimi innymi komponentami.

Kiedy zaczynamy budować stronę internetową, zazwyczaj nie wydaje się to dużym problemem, wręcz przeciwnie, wydaje się to naturalne i zgodne z zasadą DRY (Don't Repeat Yourself). Tworzymy bazowe, podstawowe style (marginesy, rozmiary czcionek,

kolory itp.), a następnie budujemy komponenty na ich podstawie, dzięki czemu nie musimy ponownie definiować tych samych reguł [5].

Problem polega na tym, że oszczędzamy czas tylko pod warunkiem, że nigdy nie zmienimy swoich bazowych reguł. W praktyce projekty stron internetowych się zmieniają. Możemy zdecydować się na zwiększenie rozmiaru czcionki nagłówków, zastosowanie innych domyślnych marginesów dla akapitów, lub że wolimy obramowania zamiast podkreśleń dla linków. Jeśli komponenty takie jak `.article-title`, `.alert-content` i `.footer-link` zależą od tych bazowych reguł, szybko zauważymy, jak kruche i powiązane są nasze style.

Jeśli komponenty zależą od bazowych stylów, zmiany w tych bazowych stylach będą wymagały sprawdzenia całej strony, aby upewnić się, że wszystko wciąż wygląda poprawnie [5].

4.2.2 Kolizje nazw klas

CSS nie ostrzega, jeśli używa się selektora klasy, który już istnieje w arkuszu stylów. Możliwość nadpisywania reguł jest jedną z cech tego języka. W rezultacie, bez wprowadzenia konwencji unikającej takich przypadków lub bez sprawdzania tego podczas budowania projektu, nie ma dobrego sposobu, aby upewnić się, że wybrana klasa, nie została już użyta przez kogoś innego.

Gdy wielu deweloperów pracuje nad tym samym kodem, szansa, że dwie osoby wybiorą tę samą nazwę klasy i nie będą o tym wiedziały, jest bardzo wysoka. Dotyczy to szczególnie powszechnych nazw, takich jak `button`, `content` czy `title`.

Nie jest to problem tylko z nazwami klas na najwyższym poziomie. Wybranie tej samej nazwy w poddrzewie może być równie niebezpieczne [5].

4.3 Rozmieszczanie klas pomiędzy plikami CSS

Jeśli style są umieszczone bezpośrednio w plikach HTML, trudno jest je ponownie wykorzystać w innych częściach strony lub w innych projektach. Tego typu kolokacja stylów prowadzi do braku wyraźnej separacji między treścią, a jej prezentacją, co utrudnia zrozumienie struktury strony.

W dużych projektach stylizacje mogą być rozproszone po wielu plikach. Na przykład, style dla przycisków mogą znajdować się w jednym pliku, dla formularzy w innym, a dla nagłówków jeszcze w innym. Odszukanie i modyfikacja stylów staje się czasochłonne, zwłaszcza gdy trzeba dokonać globalnych zmian.

Rozdział 5

Narzędzia

W celu porównania praktyk projektowania CSS stworzymy aplikację internetową korzystając, z tradycyjnego podejścia z czystym CSS, po czym zrefaktoryzujemy ją przy użyciu biblioteki Tailwind CSS i biblioteki StyleX. W tym rozdziale przedstawiamy różne narzędzia, które wykorzystaliśmy do stworzenia tej aplikacji.

5.1 Język JavaScript

Najczęściej wykorzystywanym językiem do tworzenia aplikacji internetowych jest język JavaScript. W języku JavaScript można tworzyć aplikacje internetowe w sposób bardziej skomplikowany niż w języku HTML. Wykorzystamy go w celu stworzenia serwera http, użycia bibliotek CSS in JS, oraz implementacji interfejsu użytkownika.

5.1.1 Środowisko Node.js

Node.js to darmowe, wieloplatformowe środowisko uruchomieniowe JavaScript o otwartym kodzie źródłowym, które pozwala programistom tworzyć serwery, aplikacje internetowe, narzędzia wiersza poleceń oraz skrypty. W naszym przypadku będziemy używać Node.js do uruchomienia naszej aplikacji internetowej.

5.2 React

To biblioteka dla aplikacji internetowych i natywnych. React pozwala budować interfejsy użytkownika z indywidualnych elementów zwanych komponentami. Pozwala na stworzenia własnych komponentów, takich jak `Thumbnail`, `LikeButton` oraz idocznych na listingu 5.1. Następnie połączenie ich w całe widoki, strony i aplikacje.

Listing 5.1: Kod React przedstawiający komponent Video

```
1 function Video({ video }) {
```

```
2   return (  
3     <div>  
4       <Thumbnail video={video} />  
5       <a href={video.url}>  
6         <h3>{video.title}</h3>  
7         <p>{video.description}</p>  
8       </a>  
9       <LikeButton video={video} />  
10    </div>  
11  );  
12 }
```

5.2.1 Nextjs

Next.js to framework React do budowania aplikacji internetowych typu full-stack. Używając komponentów React do budowania interfejsów użytkownika, Next.js oferuje dodatkowe funkcje i optymalizacje.

Pod maską, Next.js również abstrahuje i automatycznie konfiguruje narzędzia potrzebne do React, takie jak server http, minimalizacja, kompilacja i wiele więcej. Pozwala to skupić się na budowaniu aplikacji, zamiast tracić czas na konfigurację.

5.3 Storybook

Storybook to warsztat do budowania komponentów interfejsu użytkownika i stron w izolacji. Tysiące zespołów używa go do rozwoju interfejsu użytkownika, testowania i dokumentowania. Jest to darmowy projekt o otwartym kodzie źródłowym.

5.3.1 Chromatic

Chromatic to narzędzie do wizualnego testowania i recenzowania, które przeszukuje każdy możliwy stan interfejsu użytkownika we wszystkich przeglądarkach, aby wyłapać błędy wizualne i funkcjonalne. Wykorzystamy je do upewnienia nas, że interfejs użytkownika wygląda w pełni poprawnie i nie zawiera żadnych regresji.

Rozdział 6

Funkcjonalności bibliotek Tailwind CSS

Tailwind CSS to framework pełen klas takich jak `flex` `pt-4` `text-center`, które można komponować bezpośrednio w kodzie HTML.

6.1 Sposoby integracji

Z Tailwind CSS możemy skorzystać niezależnie od wykorzystywanego języka programowania, wystarczy wywołać komendę generującą akursz stylów w terminalu. Dzięki temu proces ten można bez problemu zautomatyzować praktycznie w każdym projekcie np. w projekcie C++ korzystającym z narzędzia takiego jak `CMake`, lub projekcie Java korzystającym z narzędzia `Gradle`, `Maven`. W naszym przypadku wykorzystaliśmy plugin `PostCSS`.

6.2 Warianty

Każda klasa może być prefiksowana dowolną ilością wariantów. Warianty sprawiają, że reguła jest nakładana tylko w odpowiednich warunkach, np. `hover:bg-red-500`, `sm: hover: bg-red-500`.

6.3 Konfigurowalność

Tailwind CSS posiada zestaw wbudowanych klas i wartości, które są w pełni konfigurowalne. Możemy tworzyć nowe klasy, warianty, zmieniać domyślne wartości istniejących klas.

6.4 Dowolne wartości

W przypadku gdy nie skonfigurowaliśmy jakiegś klasy, wariantu lub nie planujemy ich więcej używać, Tailwind CSS pozwala na użycie dowolnych wartości bez takiej potrzeby, np. `bg-[#764]`, `m-[1px]`, `[appearance:textfield]`, `group-has-[.suffix]:text-right`, `[&::-webkit-inner-spin-button]:hidden`

6.5 Selektory

6.5.1 Selektor elementów, uniwersalny, id, klasy, grupujący

Tailwind CSS nie posiada wbudowanych selektorów elementów, selektora uniwersalnego, selektora id, selektora klasy i selektorów grupujących. W tych przypadkach możemy zdefiniować reguły w pliku CSS. Możemy też stworzyć klasę wykorzystującą dowolny selektor np. selektor elementów tak jak na listingu 6.1 i 6.2. Tworząc klasę w ten sposób, musimy wykorzystać znak `&` odnoszący się do elementu, na którym znajduje się klasa, więc nie ma możliwości stworzenia klasy odnoszącej się do wszystkich elementów na stronie, chyba że umieścimy klasę na elemencie `html` będącym korzeniem drzewa HTML.

Listing 6.1: Klasa zmieniająca padding dla elementu będącego elementem `a`

```
1 <!--
2   a.\[a&\]\:p-2 {
3     padding: 0.5rem/* 8px */;
4   }
5 -->
6 <div class="[a&]:p-2"></div>
```

Listing 6.2: Klasa zmieniająca padding dla przodków będących elementem `a`

```
1 <!--
2   .\[&_a\]\:p-2 a {
3     padding: 0.5rem/* 8px */;
4   }
5 -->
6 <div class="[&_a]:p-2"></div>
```

6.5.2 Selektor potomków

Tailwind CSS posiada zestaw wbudowanych wariantów `group`, które pozwalają na stylizację potomka zależnie od stanu przodka. Dla przykładu na listingu 6.3 widzimy, klasę `group-hover:bg-red-500` która zmieni kolor tła akapitu na czerwony, gdy użytkownik najedzie na sekcję myszką. Klasa `group` sama w sobie nie nakłada żadnych stylów, jest

nakładana tylko w celu odniesienia się potomka do elementu. Istnieje też możliwość nadania grupom nazw, co może być używane w celu zagnieżdżania się grup, np. `group/item`, `group-hover/item:bg-red-500`.

Listing 6.3: Przykład wykorzystanie klasy `group`

```
1 <section class="group">
2   <div>
3     <p class="group-hover:bg-red-500">Hello world!</p>
4   </div>
5 </section>
```

6.5.3 Selektor dziecka

Tailwind CSS posiada wbudowany wariant `*`, który pozwala na stylizację dziecka. Dla przykładu na listingu 6.4 widzimy, klasę `*:bg-red-500` która zmieni kolor tła elementu `div` na czerwony.

Listing 6.4: Przykład wykorzystania selektora dziecka

```
1 <section class="*:bg-red-500">
2   <div>
3     <p>Hello world!</p>
4   </div>
5 </section>
```

6.6 Selektor młodszego rodzeństwa

Tailwind CSS posiada zestaw wbudowanych wariantów `peer`, które pozwalają na stylizację młodszego rodzeństwa zależnie od stanu starszego rodzeństwa. Dla przykładu na listingu 6.5 widzimy, klasę `peer-hover:bg-red-500` która zmieni kolor tła akapitu na czerwony, gdy użytkownik najedzie na sekcję myszką. Klasa `peer` sama w sobie nie nakłada żadnych stylów, jest nakładana tylko w celu odniesienia się młodszego rodzeństwa do elementu. Istnieje też możliwość nadania nazw, co może być używane w celu odniesienia do wielu elementów, np. `peer/item`, `peer-hover/item:bg-red-500`.

Listing 6.5: Przykład wykorzystanie klasy `peer`

```
1 <section class="peer"></section>
2 <div></div>
3 <p class="peer-hover:bg-red-500">Hello world!</p>
```

6.7 Selektor sąsiadującego młodszego rodzeństwa

Tailwind CSS nie posiada wbudowanego selektora sąsiadującego młodszego rodzeństwa. Jeżeli istnieje potrzeba jego użycia, to można posłużyć się konfiguracją w celu stworzenia wariantów analogicznie jak w przypadku selektora młodszego rodzeństwa (klasa `peer`). Lub można użyć dowolnych wartości jak w przypadku innych niewbudowanych selektorów.

6.8 pseudoklasy i pseudoelementy

Tailwind CSS posiada wbudowane warianty dla większości pseudoklas i pseudoelementów, np. `hover:bg-red`, `first:bg-red`, `first:focus:bg-red`. Możemy też posłużyć się konfiguracją w celu stworzenia nowych wariantów, np. `addVariant('hocus', ['&:hover', '&:focus'])`.

6.8.1 Funkcyjne pseudoklasy

Tailwind CSS posiada kilka wbudowanych pseudoklas, ich użycie wymaga podania dowolnej wartości, np. na listingu 6.8.1 styl elementu `div` zmieni się tylko wtedy, gdy element posiada zaznaczone pole wyboru.

```
1 <!--
2 .has-\[input\:checked\]\:p-2:has(input:checked) {
3   padding: 0.5rem/* 8px */;
4 }
5 -->
6 <div class="has-[input:checked]:p-2">...</div>
```

6.9 Style warunkowe (ang. *media query*)

Tailwind CSS posiada wbudowane warianty dla większości stylów warunkowych, np. `motion-reduce:hidden`, `dark:bg-red`. Wbudowane zostały również warianty dla popularnie wykorzystywanych szerokości ekranów, `sm` - 640px, `md` - 768px, `lg` - 1024px, `xl` - 1280px, `2xl` - 1536px. Możemy też posłużyć się konfiguracją w celu stworzenia nowych wariantów, np. `addVariant('starting-style', ['@starting-style{&}'])`.

6.10 Zmienne CSS

Aby użyć zmiennych, Tailwind CSS wymaga podania zmiennej jako dowolnej wartości, np. `bg-[var(--bg-color)]`.

6.11 Nadpisywanie stylów

Tailwind CSS nie posiada możliwości nadpisywania stylów. Nakładając dwie klasy, np. `p-2` i `p-4` nie mamy kontroli nad tym, która będzie miała większy priorytet. W takim wypadku możemy sprefixować jedną z klas znakiem `!`, np. `!p-2` aby stworzyć regułę z deklaracją `!important` w celu nadpisania stylów. Nie jest to jednak zalecane, gdyż taką operację można zastosować tylko raz i wymaga wygenerowania dodatkowego kodu CSS.

Rozdział 7

Funkcjonalności biblioteki StyleX

7.1 Przestrzenie nazw

Style muszą być tworzone przy użyciu funkcji `stylex.create`. Dzięki której możemy zdefiniować jedną lub więcej przestrzeni nazw, np. na listingu 7.1 widzimy, że zdefiniowano dwie przestrzenie nazw: `base` oraz `highlighted`. Nazwy mogą być wybrane dowolnie.

Listing 7.1: Przykład tworzenia przestrzeni nazw

```
1 import * as stylex from '@stylexjs/stylex';
2
3 const styles = stylex.create({
4   base: {
5     fontSize: 16,
6     lineHeight: 1.5,
7     color: 'rgb(60,60,60)',
8   },
9   highlighted: {
10     color: 'rebeccapurple',
11   },
12 });
```

7.2 Selektory

Biblioteka StyleX pozwala na stylizację elementów jedynie poprzez klasy, nie ma możliwości użycia reszty selektorów. Jest to intencjonalnie nałożone ograniczenie w celu stworzenia szybkiego i przewidywalnego systemu stylizacji.

7.3 pseudoklasy

Biblioteka StyleX pozwala na używanie pseudoklas, np. na listingu 7.2 widzimy, że aby zmienić kolor przycisku zależnie od stanu pseudoklas `:hover` oraz `:focus`, wartość klucza `backgroundColor` została zastąpiona obiektem pseudoklas.

Listing 7.2: Przykład wykorzystania pseudoklasy

```
1 import * as stylex from '@stylexjs/stylex';
2
3 const styles = stylex.create({
4   button: {
5     backgroundColor: {
6       default: 'lightblue',
7       ':hover': 'blue',
8       ':active': 'darkblue',
9     },
10  },
11 });
```

7.4 pseudoelementy

Biblioteka StyleX pozwala na stylizację pseudoelementów. Na listingu 7.3 widzimy, że aby zmienić kolor pseudoelementu `::placeholder` musimy umieścić jego w przestrzeni nazw.

Listing 7.3: Przykład stylizacji pseudoelementu

```
1 import * as stylex from '@stylexjs/stylex';
2
3 const styles = stylex.create({
4   input: {
5     // pseudoelement
6     '::placeholder': {
7       color: '#999',
8     },
9     color: {
10      default: '#333',
11      // pseudo-class
12      ':invalid': 'red',
13    },
14  },
15 });
```

7.5 Style warunkowe (ang. *media query*)

Biblioteka StyleX pozwala na używanie styli warunkowej, w sposób analogiczny jak używanie pseudoklas, np. na listingu 7.4 widzimy, że aby zmienić kolor przycisku zależnie od rozdzielczości ekranu, wartość klucza `backgroundColor` została zastąpiona obiektem styli.

Listing 7.4: Przykład wykorzystania styli warunkowych

```
1 import * as stylex from '@stylexjs/stylex';
2
3 const styles = stylex.create({
4   button: {
5     backgroundColor: {
6       default: 'lightblue',
7       '@media (max-width: 800px)': 'blue',
8       '@media (max-width: 1540px)': 'darkblue',
9     },
10  },
11 });
```

7.6 Zmienne CSS

Biblioteka StyleX pozwala na używanie zmiennych CSS, poprzez zastąpienie przestrzeni nazw funkcją przyjmującą wartości zmiennych jako argumenty i zwracającą przestrzeń nazw, wyglądającą tak jak dotychczas. Dla przykładu na listingu 7.5 widzimy, że wartość `height` została zastąpiona wartością dynamiczną. Używana w ten sposób funkcja nie może posiadać żadnych instrukcji poza zwracaną przestrzenią nazw, nie będzie mogła być zanalizowana na etapie kompilacji.

Listing 7.5: Przykład użycia zmiennych CSS

```
1 import * as stylex from '@stylexjs/stylex';
2
3 const styles = stylex.create({
4   bar: (height) => ({
5     height,
6   }),
7 });
8
9 function MyComponent() {
10   // The value of 'height' cannot be known at compile time.
11   const [height, setHeight] = useState(10);
12
13   return <div {...stylex.props(styles.bar(height))} />;
```

14 }

7.7 Nadpisywanie stylów

Biblioteka StyleX posiada funkcję `stylex.props` która jako argumenty przyjmuje listę przestrzeni nazw i łączy je w deterministyczny sposób ‘ostatni styl zawsze wygrywa’.

Dla przykładu na listingu 7.6 widzimy, że element `div` będzie miał fioletowy kolor, jeżeli kolejność argumentów była by odwrócona, element miałby kolor szary.

Listing 7.6: Przykład nadpisywania stylów

```
1 import * as stylex from '@stylexjs/stylex';
2
3 const styles = stylex.create({
4   base: {
5     fontSize: 16,
6     lineHeight: 1.5,
7     color: 'grey',
8   },
9   highlighted: {
10    color: 'rebeccapurple',
11  },
12 });
13
14 <div {...stylex.props(styles.base, styles.highlighted)} />;
```

Rozdział 8

Co zostało zrobione?

8.1 Projekt strony

8.1.1 Wpisy

Stworzyliśmy stronę internetową typu blog. Jak widać na listingu 8.1, stworzyliśmy pasek nawigacji, znajdujący się na każdej stronie. Ten pasek zawiera hiperłącza do strony głównej, strony wyszukiwania treści, strony logowania i strony rejestracji.

Listing 8.1: Pasek nawigacji

```
1 import { Header } from "@components/Header";
2 import Navigation from "@components/Navigation";
3 import NavigationLink from "@components/NavigationLink";
4 import { Inter } from "next/font/google";
5 import { PropsWithChildren } from "react";
6
7 const inter = Inter({ subsets: ["latin"] });
8
9 export default function Root(props: PropsWithChildren<{}>) {
10   return (
11     <div className={'bg-surface min-h-screen
12       text-on-surface ${inter.className}'}>
13       <div className={'mx-auto max-w-3xl px-4
14         sm:px-6 xl:max-w-5xl xl:px-0'}>
15         <Header>
16           <Navigation>
17             <NavigationLink href="/">Home</NavigationLink>
18             <NavigationLink href="/search">Search</NavigationLink>
19             <NavigationLink href="/login">Login</NavigationLink>
20             <NavigationLink href="/register">Register</NavigationLink>
21           </Navigation>
22         </Header>
23       <{props.children}>
```

```

24     </div>
25   </div>
26   );
27 }

```

Na listingu 8.2 widzimy, że stronie głównej znajdują się dwie sekcje z polecanymi wpisami.

Listing 8.2: Strona główna

```

1  import { ArticleData } from "@components/Article";
2  import { Articles } from "@components/Articles";
3
4  export default function HomePage({
5    articles,
6  }): {
7    articles: readonly ArticleData[];
8  }) {
9    return (
10      <main className="divide-y">
11        <Articles articles={articles.slice(0, 4)}></Articles>
12        <Articles articles={articles.slice(4)}></Articles>
13      </main>
14    );
15  }

```

Na listingu 8.3 widać, że wpisy pokazują zdjęcie, kategorię, tytuł, skrócony opis, nazwę autora, datę utworzenia i długość wpisu. Wpisy zawierają również hiperłącza do listy wpisów z tej samej kategorii, profilu autora i treści wpisu.

Listing 8.3: Wpis

```

1  export function Article({ article }: { article: ArticleData }) {
2    return (
3      <article className="text-on-surface flex flex-col
4      items-start col-span-12 gap-y-3 sm:col-span-6 xl:col-span-4">
5        <Image
6          width={500}
7          height={500}
8          alt=""
9          src={article.img}
10         className="object-cover w-full mb-2
11         overflow-hidden rounded-lg shadow-sm max-h-56"
12        />
13        {article.category && (
14          <Link href={`'/search?tag=${article.category}`}>
15            <Chip>{article.category}</Chip>
16          </Link>

```

```

17     })
18     <Link
19         className="text-lg font-bold sm:text-xl md:text-2xl"
20         href={`'/article/${article.id}`}
21     >
22         {article.title}
23     </Link>
24     <p className="text-sm">{article.description}</p>
25     <div className="pt-2 pe-0 pb-0 ps-0">
26         <Link
27             className="inline text-xs font-medium
28             mt-0 me-1 mb-0 ms-0 underline"
29             href={`'/user/${article.author.id}`}
30         >
31             {article.author.name}
32         </Link>
33         <p className="inline text-xs font-medium mt-0 me-1 mb-0 ms-1">
34             &middot;{" "}
35             {new Intl.DateTimeFormat("en", {
36                 day: "numeric",
37                 month: "short",
38                 year: "numeric",
39             }).format(article.releasedAt)}{" "}
40             &middot;
41         </p>
42         <p className="inline text-xs font-medium
43             text-on-surface/[[.38]] mt-0 me-1 mb-0 ms-1">
44             {article.length}. read
45         </p>
46     </div>
47 </article>
48 );
49 }

```

Na listingu 8.4 widać stronę pokazującą cały wpis. Warto zauważyć, że zmienna `article` przechowuje dane pochodzące z serwera. Z tego powodu jej atrybut `content` to ciąg znaków przechowujący zawartość wpisu zapisanego w języku HTML. Korzystamy więc z atrybutu `dangerouslySetInnerHTML` w bibliotece React, aby umieścić zawartość wpisu w drzewie HTML. Nie mamy tutaj możliwości dodania klas, stylów do zawartości wpisu. Korzystamy więc z selektora potomków, aby nałożyć style CSS na zawartość. W przypadku biblioteki Tailwind CSS używamy klasy `prose` stworzonej przy pomocy pluginu. Biblioteka StyleX nie posiada selektora potomków, więc musieliśmy dodać kod w języku CSS.

Listing 8.4: Strona wpisu

```

1 import { ArticleData } from "@components/Article";
2 import { BigArticle } from "@components/BigArticle";
3
4 export default function ArticlePage({ article }: { article: ArticleData }) {
5   return (
6     <main className="bg-surface text-on-surface">
7       <div className="container mx-auto py-4 grid gap-8">
8         <header>
9           <BigArticle article={article}></BigArticle>
10        </header>
11        <section
12          dangerouslySetInnerHTML={{
13            __html: article.content,
14          }}
15          className="prose py-4 lg:prose-2xl dark:prose-invert"
16        ></section>
17      </div>
18    </main>
19  );
20 }

```

8.1.2 Formularze

Formularze HTML pojawiają się praktycznie na każdej stronie internetowej. Dlatego dodaliśmy formularz logowania i rejestracji do naszej aplikacji internetowej. Na listing 8.5 widzimy przykład pola jednego z formularzy. Pola często wymagają nałożenia stylu zależnie od jednego z wielu stanów, w jakim się znajdują. Pola mogą być skupione, zawierać błędy, być puste, być automatycznie uzupełnione, być wyłączone itp. na przykładzie widać, że wykorzystano Tailwind CSS. Ze względu na ilość stanów, w jakim może znaleźć się pole formularza ilość klas i ich długość jest bardzo duża, co zmniejsza czytelność kodu. W tym przypadku lepiej byłoby oddzielić style od kodu HTML.

Listing 8.5: Pole formularza

```

1 <div className="relative">
2   <input
3     required
4     aria-invalid={props.errors?.username ? "true" : "false"}
5     name="username"
6     defaultValue={props.fields.username}
7     placeholder="John"
8     type="text"
9     className="border peer
10    disabled:border-on-surface/40
11    aria-[invalid=true]:disabled:border-on-surface/40

```

```

12     disabled:text-on-surface/40
13     disabled:placeholder-on-surface/40
14     placeholder-on-surface focus:outline-none
15     aria-[invalid=true]:border-error
16     focus:border-on-surface-variant w-full
17     pt-4 pe-4 pb-4 ps-4 mt-2 me-0 mb-0 ms-0
18     text-base block bg-surface
19     border-on-surface rounded-md"
20 />
21 <p
22     className="bg-surface pt-0
23     peer-aria-[invalid=true]:text-error
24     pe-2 pb-0 ps-2 top-0 -mt-3 me-0 mb-0
25     ms-2 font-medium peer-disabled:text-on-surface/40
26     peer-aria-[invalid=true]:peer-disabled:text-on-surface/40
27     text-on-surface
28     absolute"
29 >
30     Username
31 </p>
32 </div>
33 {!props.disabled && (
34     <span className="text-error text-sm">
35         {props.errors?.username}
36     </span>
37 )}

```

8.2 Implementacja strony

Stronę internetową tworzyliśmy w sposób iteracyjny. Zaczęliśmy od stworzenia nawigacji. Następnie stworzyliśmy historyjki, wykorzystując bibliotekę Storybook. Na przykład na listingu 8.6 widać historyjkę dla komponentu `Header`. Historyjka ta służy do interaktywnego debugowania, abyśmy mogli testować zmiany w kodzie na bieżąco.

Listing 8.6: Historyjka dla komponentu `Header`

```

1 export const Default: Story = {
2   args: {
3     children: (
4       <Navigation>
5         <NavLink href="/">Home</NavLink>
6         <NavLink href="/about">About</NavLink>
7       </Navigation>
8     ),
9   },

```

```
10   parameters: {
11     nextjs: {
12       navigation: {
13         pathname: "/",
14       },
15     },
16   },
17 };
```

Stworzyliśmy również historyjki widoczne na listingu 8.7 przedstawiające komponent z wykorzystaniem ciemnego motywu i języka czytanego od prawej do lewej. Naszym celem przy tworzeniu aplikacji było, aby cała strona wspierała różne motywy i kierunek czytania od prawej do lewej. Dzięki wykorzystaniu zmiennych CSS, implementacja motywów była bardzo prosta, niezależnie od wykorzystanych bibliotek. Zapewnienie aby strona wspierała kierunek czytania od prawej do lewej wymagało wykorzystania specyficznych atrybutów CSS, co również nie stwarzało żadnego problemu dla wykorzystanych przez nas bibliotek.

Listing 8.7: Header z wykorzystaniem ciemnego motywu i języka czytanego od prawej do lewej

```
1 export const Dark: Story = {
2   ...Default,
3   parameters: {
4     theme: "dark",
5   },
6 };
7
8 export const Rtl: Story = {
9   ...Default,
10  parameters: {
11    dir: "rtl",
12  },
13 };
```

Na listingu 8.8 widzimy, że stworzyliśmy również historyjkę z testem, który sprawdza, interakcję z nawigacją wyświetlaną na urządzeniu o małej szerokości ekranu. Na takim urządzeniu, hiperłącza są umieszczane w menu, które otwiera się lub zamyka po wciśnięciu przycisku.

Listing 8.8: Header z wykorzystaniem przełącznika nawigacji

```
1 export const Open: Story = {
2   ...Default,
3   parameters: {
4     viewport: {
5       defaultViewport: "sm",
6     },
7   },
```

```
8   play: withinGrid(async ({ canvasElement }) => {
9     const canvas = within(canvasElement);
10
11     await userEvent.click(
12       canvas.getByRole("button", {
13         name: "Open main menu",
14       })
15     );
16   }),
17   };
```

W ten sam sposób tworzyliśmy następne komponenty potrzebne to zaimplementowania kolejnych stron naszej aplikacji internetowej.

8.3 Refaktoryzacja strony

Refaktoryzując stronę internetową wykorzystywaliśmy podejście iteracyjne. Tworzyliśmy nową gałąź repozytorium git, przepisywaliśmy jeden komponent, po czym wypychaliśmy zmiany na gałąź. Każda taka zmiana uruchamia naszą aplikację Storybook na platformie Chromatic i przeprowadza testy wizualne dzięki wcześniej napisanym historyjkom. Jeżeli podczas tych testów nie wystąpiły błędy, to nasz kod jest zgodny z naszymi oczekiwaniami i przenosimy zmiany z danej gałęzi na naszą główną gałąź. W przypadku wystąpienia błędu, korzystaliśmy z możliwości debugowania na platformie Chromatic, w celu poprawienia kodu i ponownego wypchnięcia zmian.

W ten sam sposób refaktoryzowaliśmy kolejne komponenty potrzebne przez kolejne strony naszej aplikacji internetowej. Gdy wszystkie komponenty zostały zaimplementowane, mogliśmy odinstalować inne biblioteki, które nie były potrzebne w naszym projekcie.

Rozdział 9

Wnioski

W celu wyciągnięcia wniosków z napisanego przez nas kodu porównamy najczęściej wykorzystywane funkcje przy pisaniu stylów CSS według wybranej przez nas literatury: An empirical study on the use of CSS preprocessors [3] i Discovering Refactoring Opportunities in Cascading Style Sheets [2].

9.1 Zmienne

Zarówno czysty CSS jak i biblioteka Tailwind CSS i biblioteka StyleX pozwalają na użycie zmiennych.

W przypadku naszej aplikacji użyliśmy zmiennych w celu zdefiniowania wszystkich używanych kolorów. Umożliwiło nam to całkowitą zmianę wyglądu strony zależnie od zdefiniowanego przez nas motywu. Zdefiniowaliśmy motywy `light` i `dark`.

Na listingu 9.1 widzimy, że w przypadku użycia biblioteki StyleX zmienne i motywy zadeklarowaliśmy w oddzielnym pliku JavaScript. Na listingu 9.2 widzimy, że aby zastosować motyw musimy go ustawić jako wartość atrybutu `style`. Na listingu 9.3 widzimy, że aby użyć wartości zmiennych, używamy funkcji `stylex.create` aby stworzyć nową przestrzeń nazw, w której przekazujemy wartość zmiennej `surface` jako wartość atrybutu `backgroundColor`, oraz wartość zmiennej `onSurface` jako wartość atrybutu `color`. Następnie używamy funkcji `stylex.props` aby nałożyć style z przestrzeni nazw na element `div`.

Listing 9.1: StyleX definicja zmiennych CSS

```
1 import * as stylex from "@stylexjs/stylex";
2
3 export const colors = stylex.defineVars({
4   "primary": "rgb(103 80 164)",
5   "primaryContainer": "rgb(234 221 255)",
6   //...
7 });
8
```

```

9 export const dark = stylex.createTheme(colors, {
10   "primary": "rgb(208 188 255)",
11   "primaryContainer": "rgb(79 55 139)",
12   //...
13 });

```

Listing 9.2: StyleX nałożenie motywu

```

1 import { dark } from "../tokens.stylex";
2 <Wrapper style={dark}>
3   {/* ... */}
4 </Wrapper>

```

Listing 9.3: StyleX użycie zmiennych

```

1 import { colors } from "../tokens.stylex";
2
3 const styles = stylex.create({
4   root: {
5     backgroundColor: colors.surface,
6     color: colors.onSurface,
7     // ...
8   },
9   // ...
10 });
11
12 export default function Root(props: PropsWithChildren<{}>) {
13   return (
14     <div {...stylex.props(styles.root)}>
15       {/* ... */}
16     </div>
17   );
18 }

```

Na listingu 9.4 widzimy, że w przypadku użycia biblioteki Tailwind CSS rozszerzyliśmy zestaw wbudowanych kolorów `colors` o nowe kolory, takie jak `primary` oraz `primary-container`, które odnoszą się do nazwy zmiennych. Roszerzenie to pozwoliło na używanie wbudowanych klas takich jak `bg-primary-container`, `text-primary`. Stworzyliśmy również plugin pozwalający na użycie różnych motywów.

Na listingu 9.5 widzimy, że aby zastosować motyw musimy jedynie nałożyć klasę `dark`. Na listingu 9.6 widzimy, że aby użyć wartości zmiennych, używamy nowo wygenerowanych klas `bg-surface` oraz `text-on-surface`.

Listing 9.4: Tailwind CSS zmienne CSS

```

1 const themes = {
2   light: {

```

```

3      "--primary": "103 80 164",
4      "--primary-container": "234 221 255",
5      //...
6  },
7  dark: {
8      "--primary": "208 188 255",
9      "--primary-container": "79 55 139",
10     //...
11  },
12 };
13
14 /** @type {import('tailwindcss').Config} */
15 module.exports = {
16   theme: {
17     extend: {
18       colors: {
19         ...Object.fromEntries(
20           Object.keys(themes.light).map((color) => [
21             color.substring(2),
22             `rgb(var(${color}) / <alpha-value>`,
23           ])
24         ),
25       },
26     },
27   },
28   plugins: [
29     //...
30     plugin(({ addBase }) => {
31       addBase({
32         ":root": themes.light,
33         ".light": themes.light,
34         ".dark": themes.dark,
35       });
36     }),
37   ],
38 }

```

Listing 9.5: Tailwind CSS nałożenie motywu

```

1  <Wrapper className="dark">
2    {/* ... */}
3  </Wrapper>

```

Listing 9.6: Tailwind CSS użycie zmiennych

```

1  export default function Root(props: PropsWithChildren<{}>) {
2    return (

```

```

3     <div className="bg-surface text-on-surface">
4         {/* ... */}
5     </div>
6 );
7 }

```

9.2 Kolokacja kodu

Zagnieżdżanie się selektorów pozwala na zwiększenie czytelności kodu CSS, poprzez grupowanie powiązanych reguł. Na przykład na listingu 9.7 widzimy, że reguła `&.btn--primary` i `&:hover` są zagnieżdżone w regułę `.btn`. Takie zagnieżdżenie się jest równoważne z regułami o selektorach `.btn.btn--primary` oraz `.btn:hover`. Zagnieżdżając, można korzystać ze wszystkich selektorów, nie można jednak zagnieżdżać stylów warunkowych.

Listing 9.7: Zagnieżdżanie się reguł w CSS

```

1 .btn {
2     &.btn--primary {
3         color: red;
4     }
5     &:hover {
6         color: blue;
7     }
8 }
9
10 @media (min-width: 768px) {
11     .btn:hover {
12         color: purple;
13     }
14 }

```

StyleX pozwala na grupowanie reguł za pomocą przestrzeni nazw, w której korzystać można z stylów warunkowych, oraz zagnieżdżać pseudoklasy i pseudoelementy. StyleX natomiast nie pozwala na używanie reszty selektorów. Reguły CSS na listingu 9.7 wymagają napisanie podobnej ilości kodu co równoważne reguły StyleX na listingu 9.8.

Listing 9.8: Zagnieżdżanie reguł w StyleX

```

1 const styles = stylex.create({
2     btn: {
3         ':hover': 'blue',
4         '@media (min-width: 768px)': {
5             ':hover': {
6                 color: 'purple',
7             }
8         }
9     }
10 })

```

```

9   },
10  btnPrimary: {
11    color: 'red',
12  }
13 })

```

W przypadku TailwindCSS to struktura kodu HTML odpowiada za grupowanie użytych klas. Kod na listingu 9.9 wymaga najmniejszej ilości kodu. Musimy natomiast pamiętać, że kod ten musi być napisany, za każdym razem, gdy umieszczamy przycisk na stronie. Aby nie powtarzać kodu, musimy skorzystać z dowolnej biblioteki pozwalającej na tworzenie komponentów wielokrotnego użytku.

Listing 9.9: Grupowanie reguł w Tailwind CSS

```

1  <button class="hover:text-blue sm:hover:text-purple [&.primary]:text-red">
2  ...
3  </button>

```

9.3 Prefiksy

W przypadkach, gdy przeglądarki nie wspierają nowych wartości w CSS, można zdefiniować dany atrybut wielokrotnie. Na przykład na listingu 9.10 widzimy, że atrybut `position` został zdefiniowany wielokrotnie. Jeśli przeglądarka nie wspiera wartości atrybutu `sticky` to ją pominie i spróbuje zastosować wartość `-webkit-sticky` itd.

Listing 9.10: Prefiksy w CSS

```

1  .header {
2    position: fixed;
3    position: -webkit-sticky;
4    position: sticky;
5  }

```

W przypadku biblioteki StyleX można użyć funkcji `stylex.firstThatWorks` aby zdefiniować wiele wartości. Na listingu 9.11 widać style StyleX równoważne z regułami CSS na listingu 9.10.

Listing 9.11: Prefiksy w StyleX

```

1  const styles = stylex.create({
2    header: {
3      position: stylex.firstThatWorks('sticky', '-webkit-sticky', 'fixed'),
4    },
5  });

```

Biblioteka Tailwind CSS nie pozwala na zdefiniowanie jednego atrybutu wielokrotnie w deterministycznej kolejności. Sugerowany sposób na dodanie

wsparcia dla różnych przeglądarek to zastosowanie osobnej biblioteki w celu przekształcenia wynikowych stylów, np. używając biblioteki PostCSS możemy zainstalować plugin `autoprefixer`, który automatycznie dodaje prefiksy dla różnych przeglądarek. Alternatywnie możemy użyć stylów warunkowych, np. `fixed supports-[position:-webkit-sticky]:[position:-webkit-sticky] supports-[position:sticky]:sticky`

9.4 De duplikacja kodu

De duplikacja kodu jest procesem usuwania duplikatów kodu. W przypadku biblioteki StyleX, jak i Tailwind CSS obie biblioteki analizują kod źródłowy i generują optymalny kod CSS. Na przykład na listingu 9.12 widać kod przed kompilacją, a na listingu 9.13 po skompilowaniu. Wartości atrybutów zastępowane są nazwami klas. Wszystkie kolejne użycie atrybutów o identycznych wartościach w innych częściach kodu zostaną zastąpione tą samą nazwą klasy, dzięki czemu rozmiar kodu CSS się nie zmienia.

Listing 9.12: Kod przed kompilacją

```
1 const styles = stylex.create({
2   foo: {
3     color: 'red',
4     backgroundColor: 'blue',
5   },
6   bar: {
7     color: 'red',
8   },
9 });
```

Listing 9.13: Kod po skompilowaniu

```
1 const styles = {
2   foo: {
3     color: 'x1e2nbdu',
4     backgroundColor: 'x1t391ir',
5     $$css: true,
6   },
7   bar: {
8     color: 'x1e2nbdu',
9     $$css: true,
10  },
11 };
```

9.5 Nadpisywanie stylów

Język CSS ma ograniczone możliwości nadpisywania stylów. W tym celu możemy użyć selektora o większej specyficzności lub deklaracji `!important`. Oba te sposoby mogą prowadzić do nieprzewidzianych błędów w przyszłości.

Tailwind CSS nie posiada dodatkowych możliwości nadpisywania stylów, poza tymi oferowanymi przez czysty CSS.

StyleX pozwala na deterministyczne nadpisywanie stylów, ostatni nałożony styl wygrywa.

9.5.1 Porównanie kryteriów

	tradycyjny CSS	Tailwind CSS	StyleX
Zmienne	+	+	+
Kolokacja kodu	-	+	+
Prefiksy	+	+	+
De duplikacja kodu	-	+	+
Nadpisywanie stylów	-	-	+

Tabela 9.1: Kryteria

Warto zaznaczyć, że biblioteka StyleX nie pozwala na używanie pewnych selektorów języka CSS, ze względu na ich specyficzność, Tailwind CSS w pełni pozwala na użycie wszystkich selektorów.

Podsumowanie

Stworzyliśmy stronę internetową, wykorzystując tradycyjne podejście z czystym CSS, bibliotekę Tailwind CSS i bibliotekę StyleX. Zbadaliśmy najczęściej występujące problemy z kodem napisanym w języku CSS. Dokonaliśmy porównania funkcjonalności wybranych na podstawie literatury.

Bibliografia

- [1] B. Lawson and R. Sharp. *Introducing HTML5*. New Riders Publishing, Upper Saddle River, NJ, 2 edition, Oct. 2011.
- [2] D. Mazinianian and N. Tsantalis. Discovering refactoring opportunities in cascading style sheets. <https://people.ece.ubc.ca/amesbah/resources/papers/fse14.pdf>.
- [3] D. Mazinianian and N. Tsantalis. An empirical study on the use of css preprocessors. https://users.encs.concordia.ca/~nikolaos/publications/SANER_2016.pdf, 2016.
- [4] W3C. Appendix g. grammar of css 2.1. <https://www.w3.org/TR/CSS21/grammar.html>.
- [5] P. Walton. Side effects in css. <https://philipwalton.com/articles/side-effects-in-css/>, Mar. 2015.
- [6] A. Wathan. Css utility classes and "separation of concerns". <https://adamwathan.me/css-utility-classes-and-separation-of-concerns/>, Aug. 2017.